Design & Analysis of Algorithms Lecture 3

Recurrence, Analysis of control structures and few examples

Computer Science & Informatics department

Today's Lecture

- Steps in the design and analysis of an algorithm.
- Analysis of Control Structures
- Recursive calls
- Few Analysis Examples

Major Stages of Algorithm Analysis and Design

- Every algorithm needs a process in order to be created and utilized.

- FOUR steps of algorithm analysis and design:

    1. Design
    2. Analyze
    3. Implement
    4. Experiment

Stage 1: Design

- Identify the problem and thoroughly understand it.
- Consult with everybody who has an interest in the problem.
- Break out problem into stages and calculate what happens at each step so the next step can occur.
- Flowcharts and/or pseudo codes used

Stage 2: Analyze

- Analyzing how efficient the code is in solving the problem.
- Efficiency of the algorithm.
- Algorithms are measured in time and space for their efficiency.
- How your algorithm works with different **size** data structures and what kind of **time** it takes to work through those structures

Stage 3: Implement

- Writing and coding the algorithm
- Write code to execute **quickly** and handle input **data**
- Notate, comment code

Stage 4: Experiment

- Once the algorithm is designed and coded go back and **experiment** with different variables in the algorithm.
- Try to make to make it **fail**

- **Re-write** the code to work it out most efficiently.

- Keep attacking the efficiency aspect until it executes as **much** data as necessary in the **smallest** amount of time

Characteristics of a good algorithm:

1) **Precision** – the steps are precisely stated(defined).

2) **Uniqueness** – results of each step are uniquely defined and only depend on the input and the result of the preceding steps.

3) **Finiteness** – the algorithm stops after a finite number of instructions are executed.

4) **Input** – the algorithm receives input.

5) **Output** – the algorithm produces output.

6) **Generality** – the algorithm applies to a set of inputs.

Special classes of algorithms

- Order 1: $\boldsymbol{O}(1)$
- Linear $\boldsymbol{O}(n)$
- Logarithmic: $\boldsymbol{O}(\log n)$
- Quadratic $\boldsymbol{O}(n^2)$
- Polynomial $\boldsymbol{O}(n^k)$, k 1
- Exponential $\boldsymbol{O}(a^n)$, $n > 1$

Constant Time Complexity O(1)

- NO loop, NO recursion and NO call to any other non-constant time function

- i.e. set of non-recursive and non-loop statements

Constant Time Complexity O(1)

- Running time remains **constant** and does NOT depend on the size of the input.
- Execution time of an O(1) algorithm will always take the same amount of time regardless of the input **size**.

- Example of an O(1) algorithm is accessing an element in an array using an index.

Linear Time Complexity O(n)

- The loop variables are incremented/decremented by a **constant** amount.
- Running time is **proportional** to the size of the input
- Examples: Searching for an element in an unsorted array or iterating through an array and performing a constant amount of work for each element, swap function.

Quadratic Time Complexity O(n^2)

- Complexity proportional to the squared size of the input data.

- For an input of size **n**, the algorithm takes **n * n** steps to complete the operation.

- Examples: nested loop, selection sort and insertion Sort

Logarithmic Time Complexity O(Log n)

- The loop variables are **divided/multiplied** by a constant amount.

- Examples: Recursive calls in the recursive function, binary search

Logarithmic Time Complexity O(Log Log n)

- The Time Complexity of a loop is considered as O(Log Log n) if the loop variables are reduced/increased exponentially by a constant amount.

Analysis of consecutive loops?

- complexity = sum of the time complexities of individual loops.

- **Multiply** the no of iterations of each loop by the time complexity of each iteration and taking the maximum of all possible combinations.

Analysis of consecutive loops?

```
for i in range(n):
    for j in range(m):
        # some constant time operation
```

- total time complexity is O(n * m)

Comparing the asymptotic running time

- An algorithm that runs in $\boldsymbol{O}(n)$ time is better than one that runs in $\boldsymbol{O}(n^2)$ time

- Similarly, $\boldsymbol{O}(\log n)$ is better than $\boldsymbol{O}(n)$

- Hierarchy of functions:

$$\log n << n << n^2 << n^3 << 2^n$$

Complexities in the order of their growth

Complexity	*Meaning*
$O(1)$	Constant
$O(\log n)$	Logarithmic
$O(n^a)$ for $a < 1$	Sub-Linear
$O(n)$	Linear
$O(n \log n)$	Super-Linear
$O(n^a)$ for $a > 1$	Polynomial
$O(n^2)$	Quadratic
$O(n^3)$	Cubic
$O(a^n)$ for $a > 1$	Exponential
$O(n!)$	Factorial
$O(n^n)$	Extremely Fast Growing

An Example – *Linear Search*

INPUT: a sequence of *n* numbers, *key* to search for.

OUTPUT: *true* if *key* occurs in the sequence, *false* otherwise.

LinearSearch(A, *key*)	*cost*	*times*
1 $i \leftarrow 1$	c_1	1
2 **While** $i \leq n$ **and** A[i] != *key*	c_2	n
3 **do** i++	c_3	n-1
4 **if** $i \leq n$	c_4	1
5 **then return** *true*	c_5	1
6 **else return** *false*	c_6	1

So, the running time ranges between

$c_1 + c_2 + c_4 + c_5$ – **best case**

and

$c_1 + c_2(n+1) + c_3 n + c_4 + c_6$ – **worst case**

A Simple Example – *Linear Search*

INPUT: a sequence of *n* numbers, *key* to search for.

OUTPUT: *true* if *key* occurs in the sequence, *false* otherwise.

LinearSearch(A, *key*)	*cost*	*times*
1 $i \leftarrow 1$	1	1
2 **while** $i \leq n$ **and** A[i] != *key*	1	n
3 **do** i++	1	n-1
4 **if** $i \leq n$	1	1
5 **then return** *true*	1	1
6 **else return** *false*	1	1

Assign a cost of 1 to all statement executions.

Now, the running time ranges between

1+ 1+ 1 + 1 = 4 – **best case**

and

1+ (n+1)+ n + 1 + 1 = 2n+4 – **worst case**

A Simple Example – *Linear Search*

INPUT: a sequence of *n* numbers, *key* to search for.

OUTPUT: *true* if *key* occurs in the sequence, *false* otherwise.

LinearSearch(A, *key*)	*cost*	*times*
1 $i \leftarrow 1$	1	1
2 **while** $i \leq n$ **and** A[i] != *key*	1	n
3 **do** i++	1	n-1
4 **if** $i \leq n$	1	1
5 **then return** *true*	1	1
6 **else return** *false*	1	1

- If we assume that we search for a random item in the list, on an average, Statements 2 and 3 will be executed ***n*/2** times.
- Running times of other statements are independent of input. Hence, **average-case complexity** is

  $1+ n/2+ n/2 + 1 + 1 = n+3$

Order of growth

- Principal interest is to determine
  - how running time grows with input size – **Order of growth**.
  - the running time for large inputs – **Asymptotic complexity**.

- In determining the above,
  - **Lower-order terms and coefficient of the highest-order term are insignificant.**
  - **Ex: In $7n^5+6n^3+n+10$, which term dominates the running time for very large n?**

- Complexity of an algorithm is denoted by the highest-order term in the expression for running time
  - **Ex: $O(n)$, $\Theta(1)$, $\Omega(n^2)$**, etc.
  - Constant complexity when running time is independent of the input size – denoted $O(1)$.
  - **Linear Search**: **Best case Θ(1), Worst and Average cases: Θ(n).**

Analysis of Loops

- Involves finding the number of operations performed by a loop as a function of the input size.

- Step 1: Determine the number of **iterations** of the loop and the number of operations performed in each iteration.

Analysis of Loops

- no of iterations = loop control variables and loop termination condition

Analysis of Loops

- **Step 2:** Express the total number of operations performed by the loop as a function of the input size.
  - Use mathematical expressions or find a closed-form expression
- **Step 3:** Determine the order of growth of the expression for the number of operations performed by the loop.
  - Use Big O notation/dominant term

Analysis: A Harder Example

```
NESTED-LOOPS()
1  for i ← 1 to n
2  do
3      for j ← 1 to 2i
4      do k = j ...
5          while (k ≥ 0)
6          do k = k − 1 ...
```

Solution

- How do we analyze the running time of an algorithm that has complex nested loop?
- The answer write out the loops as summations and then solve the summations.
- To convert loops into summations, we work from inside-out.

```
NESTED-LOOPS()
1  for i ← 1 to n
2  do
3      for j ← 1 to 2i
4      do k = j ...
5          while (k ≥ 0)
6          do k = k − 1 ...
```

Analysis: A Harder Example

```
NESTED-LOOPS()
1  for i ← 1 to n
2  do for j ← 1 to 2i
3       do k = j
4            while (k ≥ 0)
5            do k = k − 1
```

$$I(j) = \sum_{k=0}^{j} 1 = j + 1$$

It is executed for k = j, j − 1, j − 2, . . . , 0. Time spent inside the while loop is constant. Let I() be the time spent in the while loop

Analysis: A Harder Example

middle for loop.

```
NESTED-LOOPS()
1  for i ← 1 to n
2  do for j ← 1 to 2i
3       do k = j
4          while (k ≥ 0)
5          do k = k − 1
```

Its running time is determined by i. Let M() be the time spent in the for loop:

$$
\begin{aligned}
M(i) &= \sum_{j=1}^{2i} I(j) \\
&= \sum_{j=1}^{2i} (j+1) \\
&= \sum_{j=1}^{2i} j + \sum_{j=1}^{2i} 1 \\
&= \frac{2i(2i+1)}{2} + 2i \\
&= 2i^2 + 3i
\end{aligned}
$$

Analysis: A Harder Example

```
NESTED-LOOPS()
1  for i ← 1 to n
2  do for j ← 1 to 2i
3       do k = j
4            while (k ≥ 0)
5            do k = k − 1
```

Finally the *outer-most for* loop

Let T() be running time of the entire algorithm:

$$
\begin{aligned}
T(n) &= \sum_{i=1}^{n} M(i) \\
&= \sum_{i=1}^{n} (2i^2 + 3i) \\
&= \sum_{i=1}^{n} 2i^2 + \sum_{i=1}^{n} 3i \\
&= 2\frac{2n^3 + 3n^2 + n}{6} + 3\frac{n(n+1)}{2} \\
&= \frac{4n^3 + 15n^2 + 11n}{6} \\
&= \Theta(n^3)
\end{aligned}
$$

Analyzing Control Structures Summary

- Algorithm usually proceeds from the inside out
- First determine the time required by individual instructions
- Second, combine the times according to the control structures that combine the instructions in the program
- Some control structures sequencing are easy to evaluate
- Others such as while loops are more difficult

Analyzing Control Structures Sequencing

- A sequence is a series of statements that do not alter the execution path within an algorithm.
- Statements such as assign and add are sequence statements.
- A call to another algorithm is also considered a sequence statement.
- Selection statements evaluate one or more alternatives. Paths are followed based on its result.

Analyzing Control Structures Sequencing

- Let P_1 and P_2 be two fragments of an algorithm
- Let t_1 and t_2 be the times taken by P_1 and P_2 respectively

Sequencing Rule

- The time required to compute " P_1 **:** P_2 ", is simply t_1+ t_2.
- By the maximum rule, this time is in (max(t_1, t_2))
- It could happen that one of the parameters that control t_2 depend on the result of the computation performed by P_1
- Thus analysis of "P_1 **:** P_2" cannot always be performed by considering P_1 and P_2 independently

Analyzing Control Structures: "For" loops

Consider the loop

- **for** i ← 1 **to** m **do** P(i)

- Suppose the loop is part of a larger algorithm working on an instance of size *n.* Let *t* denote the time required to compute *P(i)*

- *P(i)* is performed *m* times, each time at a cost of *t* Total time required by the loop is I = *mt*

  If the time *t(i)* required for *P(i)* varies as a function of *i*, the loop takes a time given by the sum

$$I = \sum_{i=1}^{m} t(i)$$

Analyzing Control Structures

for i ← 1 **to** m **do** P(i)

* Time needed for loop control must also be taken into account. For loop can be like:

i ← 1	: c
while i ≤ m do	: (m+1)c
P(i)	: mt
i ← i + 1	: mc
for sequencing operations(go to)	: mc

$I \leq (c + (m+1)c+mt+mc+mc)$

$I \leq (t+3c)m+2c)$

Analysing control structures: for loops

* Where c can be upper bound on the time required by each of the above mentioned operations. If c is negligible compared to t,

$$I \simeq mt$$

Analysing of For Loop for Computing

Function Fibiter(n)

```
{ Calculates the n-th term of the Fibonacci sequence}
    i ← 1;
    j ← 0
    for k ← 1 to n do
        j ← i + j
        i ← j – i
    return j
```

Fibonacci (5)

i = 1

j = 0

k =	**1**	**2**	**3**	**4**	**5**
j =	**1**	**1**	**2**	**3**	**5**
i =	**0**	**1**	**1**	**2**	**3**

- Time taken by the instructions inside the for loop is bounded above constant c
- Time taken by the for loop = nc
- Algorithm takes a time in O(n), Ω (n) and θ(n)
- Length of integer is important to determine the time taken by kth trip as the values of i & j are f_{k-1} and f_k respectively.

While and Repeat Loops

Using a Barometer

- A barometer instruction is one that is executed at least as often as any other instruction in the algorithm so it simplifies the analysis procedure
- The time taken by the entire algorithm is in the exact order of the number of times that the barometer instruction is executed
- Barometer approach avoids the need to introduce constants such as those bounding the time taken by various elementary operations
- Consider the function Fibiter(n)

Function Fibiter (n)

```
i ← 1; j ← 0
for k ←  1 to n do j ←  i+j
                  i ← j – l
return j
```

While and Repeat Loops

- The instructions $j \leftarrow i + j$ can be taken as barometer as it is executed n times. So the algorithm takes a time in $\theta(n)$

- When an algorithm involves several nested loops, any instruction of the innermost loop can usually be used as barometer.
- The use of barometer is a handy tool to simplify the analysis of algorithms but must be used with care

Difference in Analysis

```
for(i=0;i<m;i++)

 for(j=0;j<n;j++)

  for(k=0;k<q;k++)
 {    }
```

$O(n^3)$

```
for(i=0;i<m;i++)
   { }
 for(j=0;j<n;j++)
   {   }
  for(k=0;k<q;k++)
    {    }
```

$O(n)$

RECURRENCE

What is a recurrence relation?

- A recurrence relation, T(n), is a recursive function of integer variable **n**.
- Like all recursive functions, it has both recursive case and base case.
- Example:

$$T(n) = \begin{cases} a & \text{if } n = 1 \\ 2T(n/2) + bn + c & \text{if } n > 1 \end{cases}$$

- The portion of the definition that does not contain T is called the **base case** of the recurrence relation
- The part that contains T is called the **recurrent or recursive case**.

Forming Recurrence Relations

- For a given recursive method, the base case and the recursive case of its recurrence relation correspond directly to the base case and the recursive case of the method.
- Example 1: Write the recurrence relation for the following method.

```
public void f (int n) {
    if (n > 0) {
        System.out.println(n);
        f(n-1);
    }
}
```

- The base case is reached when n == 0. The method performs one comparison. Thus, the number of operations when n == 0, T(0), is some constant a.
- When n > 0, the method performs two basic operations and then calls itself, using ONE recursive call, with a parameter n – 1.
- Therefore the recurrence relation is:

$$T(0) = a \qquad \text{for some constant a}$$
$$T(n) = b + T(n-1) \qquad \text{for a constant b}$$

Forming Recurrence Relations

Example 2: Write the recurrence relation for the following method.

```
public int g(int n) {
   if (n == 1)
      return 2;
   else
      return 3 * g(n / 2) + g( n / 2) + 5;
}
```

- The base case is reached when n == 1. The method performs one comparison and one return statement. Therefore, T(1), is constant **c**.
- When **n > 1**, the method performs **TWO** recursive calls, each with the parameter n ***/ 2,*** and some constant # of basic operations.
- Hence, the recurrence relation is:

$$T(1) = c \qquad \text{for some constant } c$$

$$T(n) = b + 2T(n / 2) \qquad \text{for a constant } b$$

Recursive vs Non-recursive algorithms

- A recursive function in general has an extremely **high time complexity** while a non-recursive one does not.
- A recursive function generally has **smaller code size** whereas a non-recursive one is larger.
- In some situations, only a recursive function can perform a specific task, but in other situations, **both** a recursive function and a non-recursive one can do it.

Solving Recurrence Relations

- ✓ Solving a recurrence relation means obtaining a closed-form solution .

- ✓ There are four methods to solve recurrence relations that represent the running time of recursive methods:
  - Iteration method (*unrolling and summing)*
  - Substitution method
  - Recursion tree method
  - Master method

Iteration method (*unrolling and summing)*

Solving Recurrence Relations - Iteration method- Useful Formulae

- Steps:
  - Expand the recurrence
  - Express the expansion as a summation by plugging the recurrence back into itself until you see a pattern.
  - Evaluate the summation
- In evaluating the summation one or more of the <u>following summation formulae</u> may be used:
- <u>1. Arithmetic series:</u>

$$\sum_{k=1}^{n} k = 1 + 2 + \ldots + n = \frac{n(n+1)}{2}$$

- <u>2. Geometric Series:</u>

$$\sum_{k=0}^{n} x^k = 1 + x + x^2 + \ldots + x^n = \frac{x^{n+1} - 1}{x - 1} (x \neq 1)$$

$$\sum_{k=0}^{n-1} x^k = \frac{x^n - 1}{x - 1} (x \neq 1)$$

•Special Cases of Geometric Series:

$$\sum_{k=0}^{n-1} 2^k = 2^n - 1$$

Recap

$$\sum_{k=0}^{\infty} x^k = \frac{1}{1 - x} \quad \text{if } x < 1$$

Solving Recurrence Relations - Iteration method- Useful Formulae

Recap

- 3. Harmonic Series:

$$\sum_{k=1}^{n} \frac{1}{k} = 1 + \frac{1}{2} + \ldots + \frac{1}{n} \approx \ln n$$

- 4. Others:

$$\sum_{k=1}^{n} \lg k \approx n \lg n$$

$$\sum_{k=1}^{n} k^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\sum_{k=0}^{n-1} c = cn.$$

$$\sum_{k=0}^{n} k(k+1) = \frac{n(n+1)(n+2)}{3}$$

$$\sum_{k=0}^{n-1} \frac{1}{2^k} = 2 - \frac{1}{2^{n-1}}$$

Analysis Of Recursive Binary Search

```
public int binarySearch (int target, int[] array,
                         int low, int high) {
   if (low > high)
      return -1;
   else {
      int middle = (low + high)/2;
      if (array[middle] == target)
         return middle;
      else if(array[middle] < target)
         return binarySearch(target, array, middle + 1, high);
      else
         return binarySearch(target, array, low, middle - 1);
     }
}
```

- The recurrence relation for the running time of the method is:

$T(1) = a$ if $n = 1$ (one element array)

$T(n) = T(n / 2) + b$ if $n > 1$

Analysis Of Recursive Binary Search

– **Expanding:**

$T(n) = T(n / 2) + b$

$= [T(n / 4) + b] + b = T(n / 2^2) + 2b$

$= [T(n / 8) + b] + 2b = T(n / 2^3) + 3b$

$= \ldots\ldots\ldots$

$= T(n / 2^k) + kb$

– **When $n / 2^k = 1 \rightarrow n = 2^k \rightarrow k = \log_2 n$, we have:**

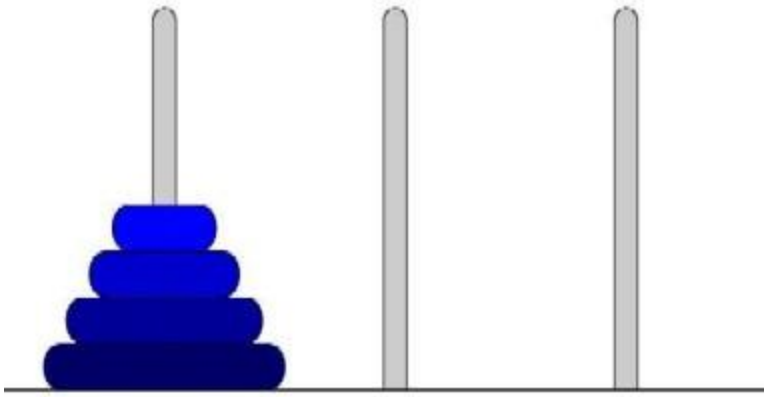
$T(n) = T(1) + b \log_2 n$

$= a + b \log_2 n$

– Therefore, Recursive Binary Search is **O(log n)**

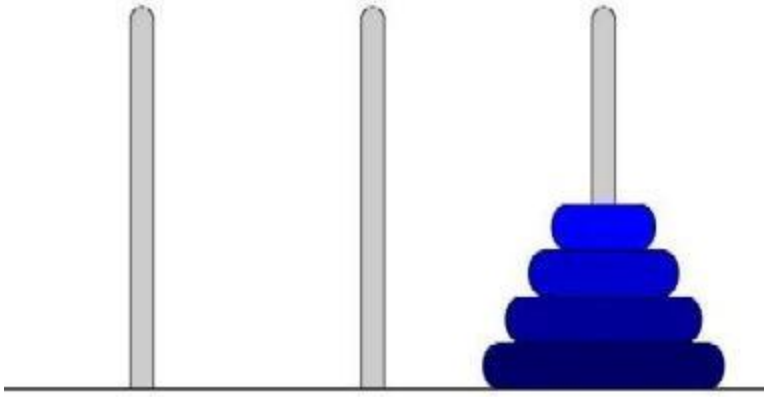
Tower of Hanoi

- Tower of Hanoi is a mathematical puzzle invented by a French Mathematician Edouard Lucas in 1883.
- The game starts by having few discs stacked in increasing order of size. The number of discs can vary, but there are only three pegs.



Tower of Hanoi

- The Objective is to **transfer** the entire tower to one of the other pegs. However you can only move one disk at a time and you can never stack a larger disk onto a smaller disk. Try to solve it in fewest possible moves.



Tower of Hanoi

Solution

To get a better understanding for the general algorithm used to solve the Tower of Hanoi, try to solve the puzzle with a small amount of Disks, 3 or 4, and once you master that , you can solve the same puzzle with more discs with the following algorithm.

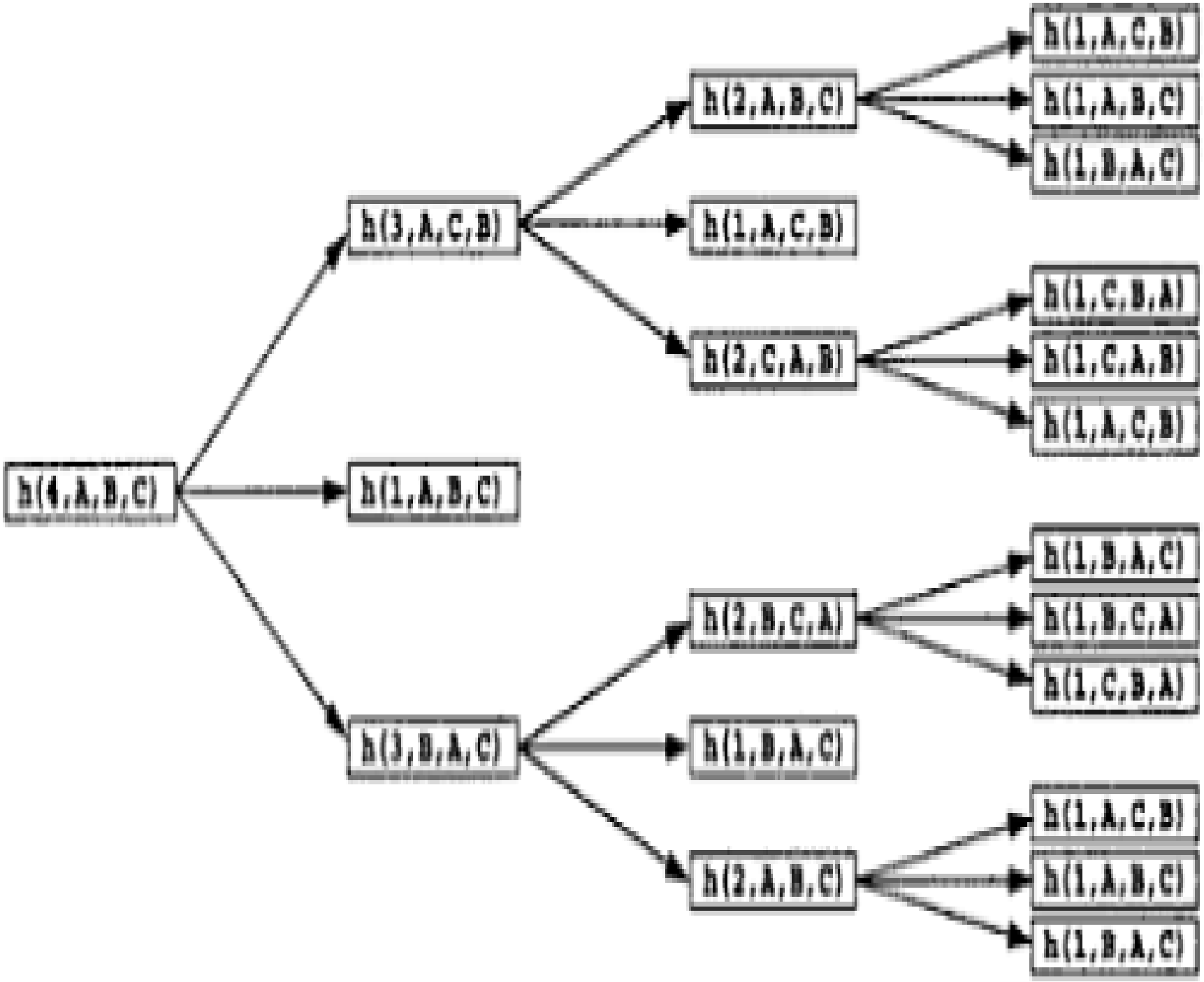
Tower of Hanoi

Recursive Solution for the Tower of Hanoi with algorithm

```
public static void hanoi(int n, char BEG, char AUX, char END)
{
   if (n == 1)
      System.out.println(BEG + " --------> " + END);
     else
 {
      hanoi(n - 1, BEG, END, AUX);

      System.out.println(BEG + " --------> " + END);

      hanoi(n - 1, AUX, BEG,END);
     }
   }
```


h(4,A,B,C)
h(3,A,C,B)
h(1,A,B,C)
h(3,B,A,C)
h(2,A,B,C)
h(1,A,C,B)
h(2,C,A,B)
h(2,B,C,A)
h(1,B,A,C)
h(2,A,B,C)
h(1,A,C,B)
h(1,A,B,C)
h(1,B,A,C)
h(1,C,B,A)
h(1,C,A,B)
h(1,A,C,B)
h(1,B,C,A)
h(1,B,C,A)
h(1,C,B,A)
h(1,A,C,B)
h(1,A,B,C)
h(1,B,A,C)

Tower of Hanoi

- **Explicit Pattern**

Number of Disks	Number of Moves
1	1
2	3
3	7
4	15
5	31

- *Powers of two help reveal the pattern:*

Number of Disks (n)	Number of Moves
1	2^1 - 1 = 2 - 1 = **1**
2	2^2 - 1 = 4 - 1 = **3**
3	2^3 - 1 = 8 - 1 = **7**
4	2^4 - 1 = 16 - 1 = **15**
5	2^5 - 1 = 32 - 1 = **31**

Analysis Of Recursive Towers of Hanoi Algorithm

```
public static void hanoi(int n, char BEG, char AUX, char END){
   if (n == 1)
      System.out.println(from + " --------> " + to);
   else{
      hanoi(n - 1, BEG, END, AUX);
      System.out.println(from + " --------> " + to);
      hanoi(n - 1, END, AUX, BEG);
   }
}
```

- The recurrence relation for the running time of the method **hanoi** is:

$$T(n) = a \qquad \text{if } n = 1$$

$$T(n) = 2T(n - 1) + b \qquad \text{if } n > 1$$

Analysis Of Recursive Towers of Hanoi Algorithm

Expanding:

$T(n) = 2T(n-1) + b$

$= 2[2T(n-2) + b] + b = 2^2 T(n-2) + 2b + b$

$= 2^2 [2T(n-3) + b] + 2b + b = 2^3 T(n-3) + 2^2 b + 2b + b$

$= 2^3 [2T(n-4) + b] + 2^2 b + 2b + b = 2^4 T(n-4) + 2^3 b + 2^2 b + 2^1 b + 2^0 b$

$= \ldots\ldots$

$= 2^k T(n-k) + b[2^{k-1} + 2^{k-2} + \ldots 2^1 + 2^0]$

$$= 2^k T(n-k) + b\sum_{i=0}^{k-1} 2^i$$

$$= 2^k T(n-k) + b(2^k - 1)$$

$$\sum_{k=0}^{n-1} 2^k = 2^n - 1$$

Géométric Séries

When k = n – 1, we have:

$$T(n) = 2^{n-1} T(1) + b(2^{n-1} - 1)$$
$$= (a+b)\, 2^{n-1} - b$$
$$= \left(\frac{a+b}{2}\right) 2^n - b$$

Therefore, The method **hanoi** is **$O(2^n)$**